Program 3: Maven Project To Connect To Web Servers Using Selenium Dependency

Before connecting to web servers, ChromeDriver must be installed. Check the Google Chrome version:

google-chrome --version

Example output: Google Chrome 133.0.6943.141

Extract the Chrome version and store it in a variable:

CHROME_VERSION=$(google-chrome --version | awk '{print $3}' | cut -d'.' -f1-3)

Install jq (used to process JSON data):

sudo apt install jq

If curl is not installed, install it using:

sudo apt install curl

Get the latest compatible ChromeDriver version:

LATEST_DRIVER=$(curl -s "https://googlechromelabs.github.io/chrome-for-testing/latest-patch-versions-per-build.json" | jq -r ".builds["$CHROME_VERSION"].version")

Download ChromeDriver:

wget https://storage.googleapis.com/chrome-for-testing public/$LATEST_DRIVER/linux64/chromedriver-linux64.zip

Extract the downloaded file:

unzip chromedriver-linux64.zip

Move ChromeDriver to the system path:

sudo mv chromedriver-linux64/chromedriver /usr/local/bin/chromedriver

Give execution permission:

sudo chmod +x /usr/local/bin/chromedriver

Verify the installation:

chromedriver --version

The output version should match the installed Chrome version.

Step 1: create a maven project on terminal

mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenSeleniumApp01 - DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false

```
shweta@shweta-VirtualBox:~/Devops$ mvn archetype:generate -DgroupId=com.example -DartifactId=MyMave
nSeleniumApp01 -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
[INFO] Scanning for projects...
[INFO]
[INFO] ------------------< org.apache.maven:standalone-pom >-------------------
[INFO] Building Maven Stub Project (No POM) 1
[INFO] --------------------------------[ pom ]---------------------------------
[INFO]
[INFO] >>> maven-archetype-plugin:3.4.1:generate (default-cli) > generate-sources @ standalone-pom
>>>
[INFO]
[INFO] <<< maven-archetype-plugin:3.4.1:generate (default-cli) < generate-sources @ standalone-pom
<<<
[INFO]
[INFO]
[INFO] --- maven-archetype-plugin:3.4.1:generate (default-cli) @ standalone-pom ---
[INFO] Generating project in Batch mode
Downloading from central: https://repo.maven.apache.org/maven2/archetype-catalog.xml
Downloaded from central: https://repo.maven.apache.org/maven2/archetype-catalog.xml (18 MB at 7.7 M
B/s)
[INFO] No archetype defined. Using maven-archetype-quickstart (org.apache.maven.archetypes:maven-ar
chetype-quickstart:1.0)
[INFO] ----------------------------------------------------------------------------
[INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-qu
ickstart:1.0
[INFO] ----------------------------------------------------------------------------
[INFO] Parameter: basedir, Value: /home/shweta/Devops
[INFO] Parameter: package, Value: com.example
```

Step 2: Edit the pom.xml file.

Add the Selenium dependency by going to the Maven Central Repository, searching for Selenium, selecting **Selenium Java**, choosing the latest version, and copying the dependency code. Paste this dependency into the pom.xml file.

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/maven-v4_0_0.xsd">

<modelVersion>4.0.0</modelVersion>

<groupId>com.example</groupId>

<artifactId>MyMavenSeleniumApp01</artifactId>

<packaging>jar</packagi<ng>

<version>1.0-SNAPSHOT</version>

<name>MyMavenSeleniumApp01</name>

<url>http://maven.apache.org</url>

<dependencies>

<dependency>

<groupId>junit</groupId>

<artifactId>junit</artifactId>

<version>3.8.1</version>

<scope>test</scope>

</dependency>

<!-- https://mvnrepository.com/artifact/org.seleniumhq.selenium/selenium-java -->

<dependency>

<groupId>org.seleniumhq.selenium</groupId>
```

```
<artifactId>selenium-java</artifactId>
<version>4.29.0</version>
</dependency>
</dependencies>
<!-- Build: Configuring plugins and build settings -->
<build>
<plugins>
<!-- Example: Maven Compiler Plugin to compile Java code -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler-plugin</artifactId>
<version>3.8.1</version>
<configuration>
<source>1.7</source>
<target>1.7</target>
</configuration>
</plugin>
<!-- Example: Maven Surefire Plugin to run tests -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
```

<artifactId>maven-surefire-plugin</artifactId>

<version>2.22.2</version>

</plugin>

<plugin>

<groupId>org.apache.maven.plugins</groupId>

<artifactId>maven-jar-plugin</artifactId>

<version>3.0.1</version>

<configuration>

<archive>

<manifestEntries>

<Main-Class>com.example.App</Main-Class>

</manifestEntries>

</archive>

</configuration>

</plugin> </plugins> </build> </project>

Step 3: Before writing the App.java file, inspect the SauceDemo webpage to find the element IDs required for automation.

Right-click on the webpage and select **Inspect**. The HTML code will be displayed. Click on the cursor icon (arrow symbol) available on the top-left of the inspect panel, then hover over the username field to identify its ID, which is **user-name**.

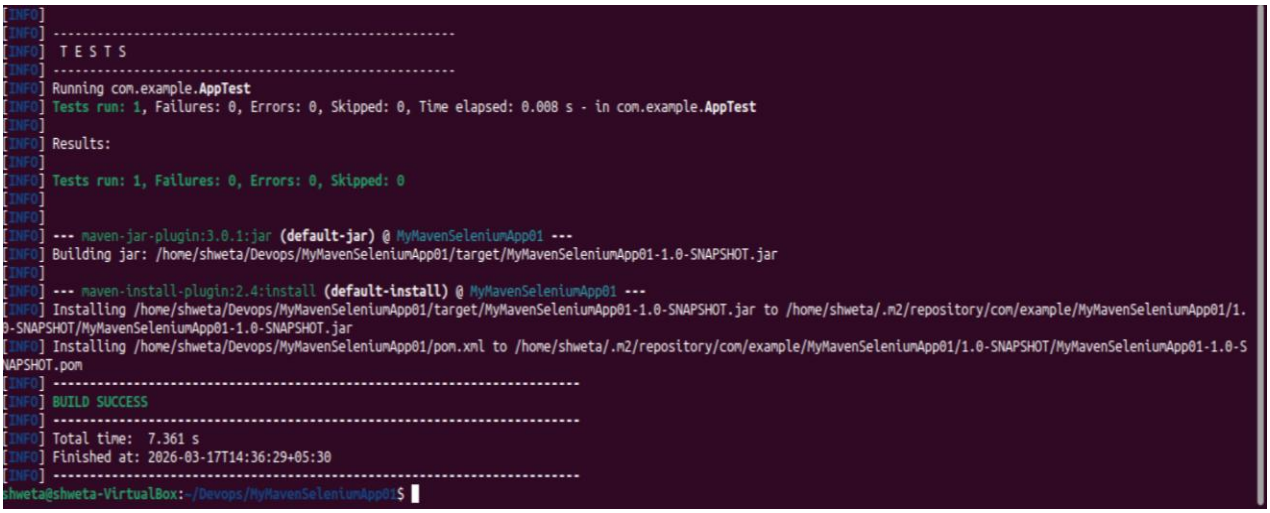
Similarly, check the password field (ID: **password**) and the login button (ID: **login-button**). Make a note of these IDs, as they will be used in the App.java file.

App.java for selenium project

```
package com.example;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
public class App
{
   public static void main( String[] args )
   {
      WebDriver driver=new
      ChromeDriver();
      driver.get("https://www.saucedemo.com
      /");
      driver.manage().window().maximize();

      driver.findElement(By.id("user-
      name")).sendKeys("standard_user");
      driver.findElement(By.id("password")).
      sendKeys("secret_sauce");
      driver.findElement(By.id("login-
      button")).click();
   }}
```

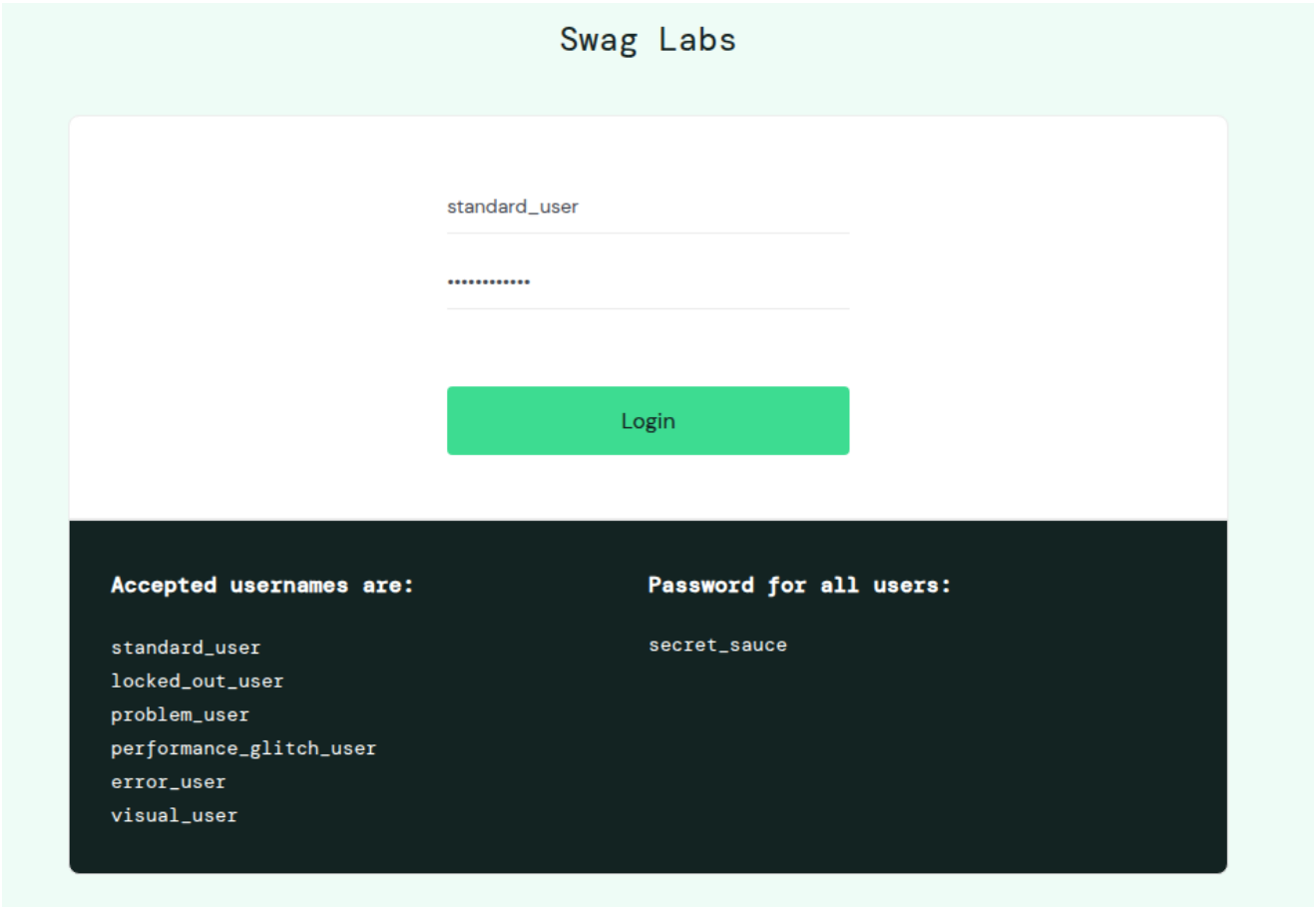
Step 4: Build and run the maven application

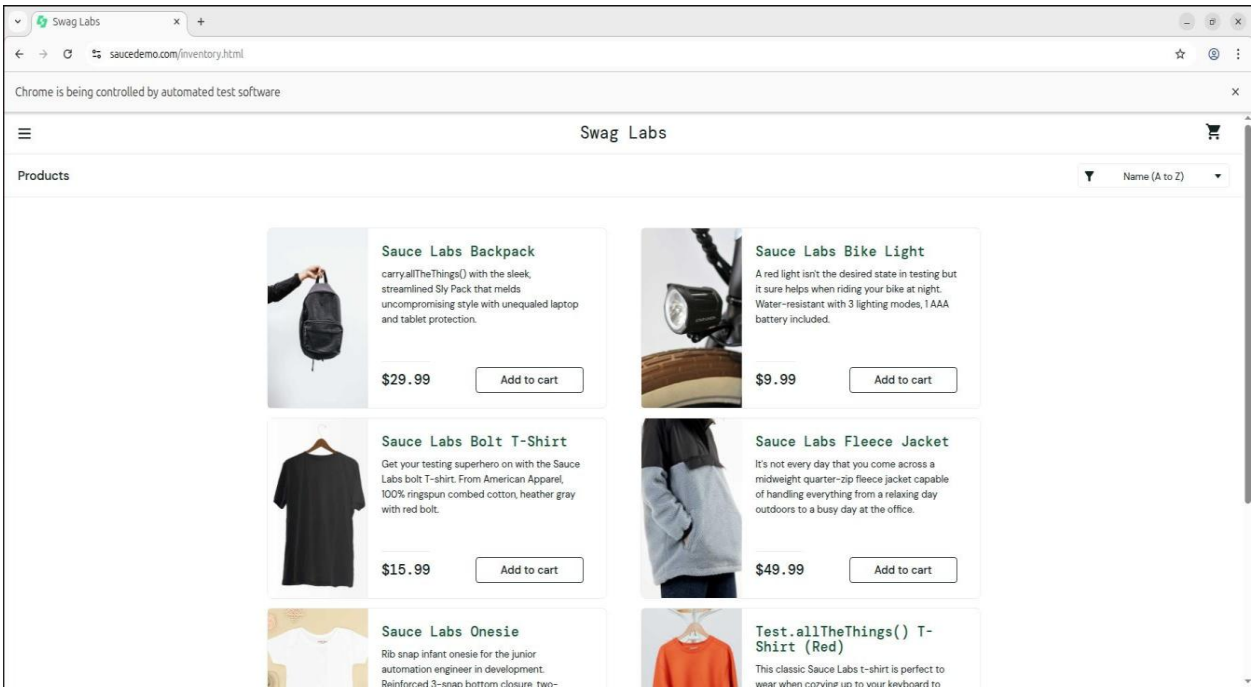
mvn clean install



Step 5: run the below command

mvn exec:java -Dexec.mainClass="com.example.App"


Swag Labs
standard_user
Login
Accepted usernames are:
standard_user
locked_out_user
problem_user
performance_glitch_user
error_user
visual_user
Password for all users:
secret_sauce


Swag Labs
saucedemo.com/inventory.html
Chrome is being controlled by automated test software
Swag Labs
Products
Name (A to Z)
Sauce Labs Backpack
carry.allTheThings() with the sleek, streamlined Sly Pack that melds uncompromising style with unequaled laptop and tablet protection.
$29.99
Add to cart
Sauce Labs Bike Light
A red light isn't the desired state in testing but it sure helps when riding your bike at night. Water-resistant with 3 lighting modes, 1 AAA battery included.
$9.99
Add to cart
Sauce Labs Bolt T-Shirt
Get your testing superhero on with the Sauce Labs bolt T-shirt. From American Apparel, 100% ringspun combed cotton, heather gray with red bolt.
$15.99
Add to cart
Sauce Labs Fleece Jacket
It's not every day that you come across a midweight quarter-zip fleece jacket capable of handling everything from a relaxing day outdoors to a busy day at the office.
$49.99
Add to cart
Sauce Labs Onesie
Rib snap infant onesie for the junior automation engineer in development. Reinforced 3-snap bottom closure, two-
Test.allTheThings() T-Shirt (Red)
This classic Sauce Labs t-shirt is perfect to wear when cozying up to your keyboard to

Sauce Demo Website

Sauce Demo (https://www.saucedemo.com/) is a fake e-commerce website created by Sauce Labs for testing and learning web automation tools like Selenium.

It is used for:

- Writing automation scripts and UI tests
- Demonstrating cloud testing features
- Learning and practicing test automation

Pushing MyMavenSeleniumApp01 to Github:

```
shweta@shweta-VirtualBox:~/Devops/MyMavenSeleniumApp01$ git init
Reinitialized existing Git repository in /home/shweta/Devops/MyMavenSeleniumApp01/.git/
shweta@shweta-VirtualBox:~/Devops/MyMavenSeleniumApp01$ git add .
shweta@shweta-VirtualBox:~/Devops/MyMavenSeleniumApp01$ git commit -m "first commit"
[master 5ca395f] first commit
 3 files changed, 4 insertions(+), 4 deletions(-)
shweta@shweta-VirtualBox:~/Devops/MyMavenSeleniumApp01$ git remote add origin https://github.com/Shweta311204/MyMavenSeleniumApp.git
error: remote origin already exists.
shweta@shweta-VirtualBox:~/Devops/MyMavenSeleniumApp01$ git remote add origin https://github.com/Shweta311204/MyMavenSeleniumApp01.git
error: remote origin already exists.
shweta@shweta-VirtualBox:~/Devops/MyMavenSeleniumApp01$ git remote set-url origin https://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@github.com/Shweta311204/MyMavenSelenium
App01.git
shweta@shweta-VirtualBox:~/Devops/MyMavenSeleniumApp01$ git push -u origin master
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 3 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (7/7), 875 bytes | 437.00 KiB/s, done.
Total 7 (delta 5), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (5/5), completed with 5 local objects.
To https://github.com/Shweta311204/MyMavenSeleniumApp01.git
   c82859f..5ca395f  master -> master
Branch 'master' set up to track remote branch 'master' from 'origin'.
shweta@shweta-VirtualBox:~/Devops/MyMavenSeleniumApp01$
```

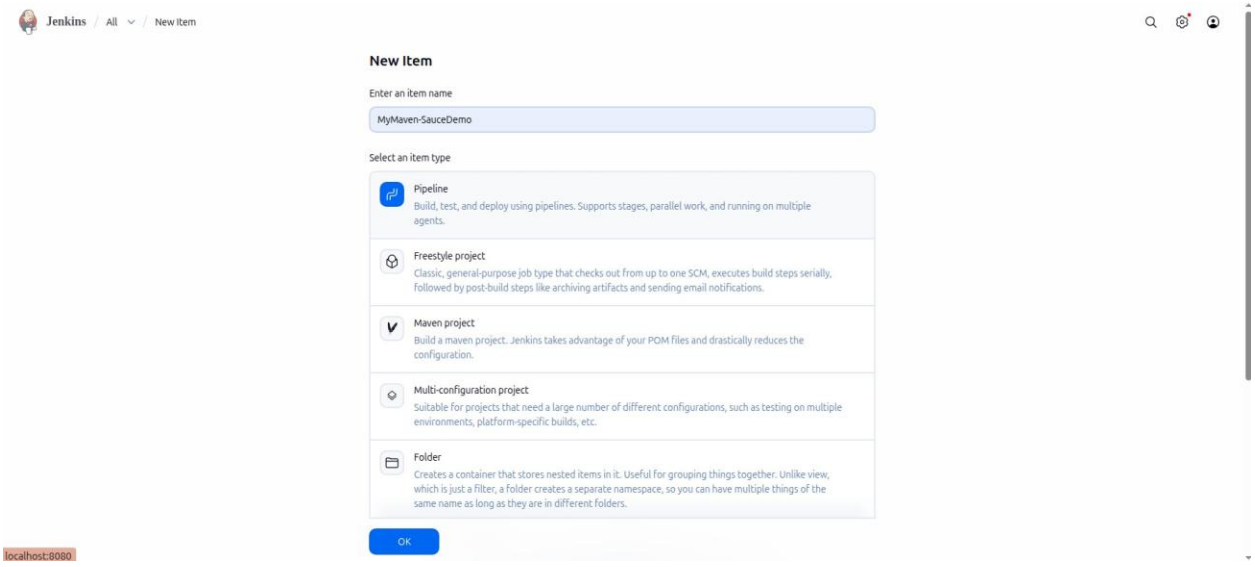
Setting Up Jenkins Pipeline for MyMavenSeleniumApp01

Step 1: To deploy **MyMavenSeleniumApp01**, open the terminal, cd into the **MyMavenSeleniumApp01** folder, create a **Jenkinsfile** containing the pipeline code, and then commit and push it to GitHub.

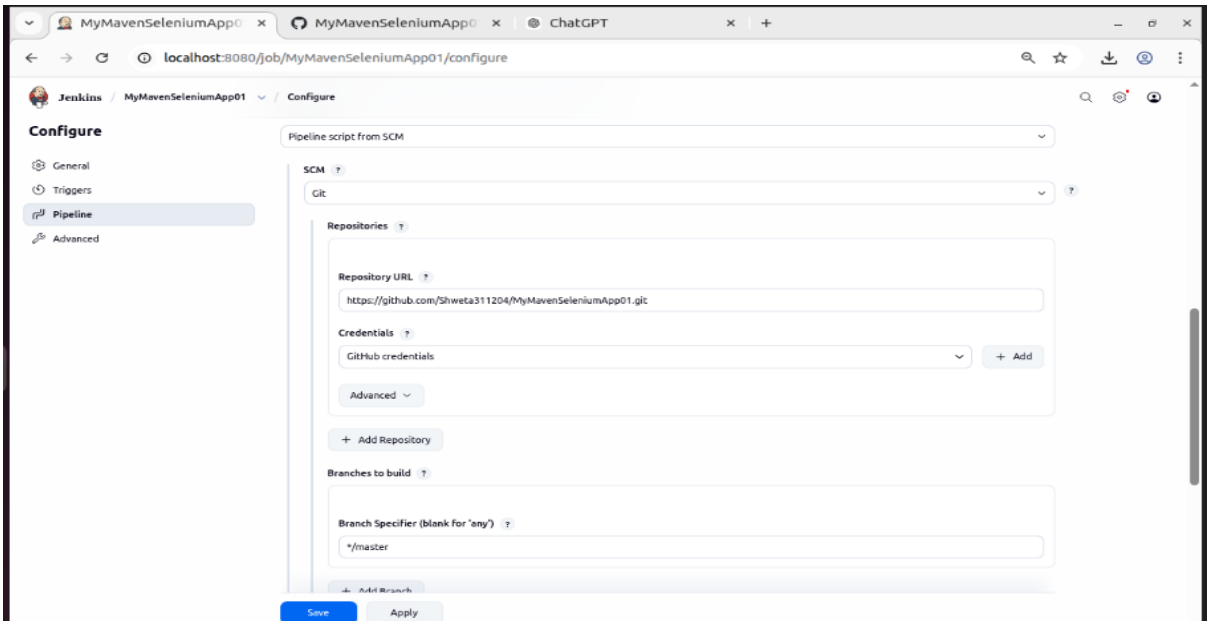
```
pipeline {

agent any  // Use any available agent tools {

maven 'Maven'  // Ensure this matches the name configured in Jenkins
}

stages {

stage('Checkout') { steps {

git branch: 'master', url:
'https://github.com/Shweta311204/MyMavenSeleniumApp01.git'
}
}

stage('Build') { steps {

sh 'mvn clean package'  // Run Maven build
}
}

stage('Test') { steps {

sh 'mvn test'  // Run unit tests
}
}

stage('Run Application') { steps {

// Start the JAR application

sh 'mvn exec:java -Dexec.mainClass="com.example.App"'
}
}
}

post {

success {
```

```
echo 'Build and deployment successful!'
}

failure {

echo 'Build failed!'
}
}
}
```

Step 2: To deploy **MyMavenSeleniumApp01**, you first need to create a Jenkins pipeline job that will use the Jenkinsfile from your project repository. Then, go to the Jenkins home page and click on **"New Item"** to create a job, enter the item name (e.g., *MyMavenSeleniumApp01*), and select **Pipeline** as the project type. This option allows you to build, test, and deploy applications using a Jenkinsfile. After selecting the pipeline type, click **"OK"** to proceed to the job configuration page.



Step 3: In **MyMavenSeleniumApp01**, go to the **Pipeline** section and under **Definition**, select **Pipeline script from SCM**. Choose **Git** as the SCM, and under **Repository URL**, paste the GitHub URL of **MyMavenSeleniumApp01**. In the **Credentials** field, select the previously created credential (e.g., *GitHub Credential*). Finally, in **Script Path**, enter the name of the Jenkinsfile you created in the MyMavenSeleniumApp01 folder and pushed to GitHub. Then click **Apply** and **Save** to finalize the configuration.



Step 4: After clicking **Apply and Save**, go back to **MyMavenSeleniumApp01 in Jenkins** and click
Build Now. If you get an error like:

java.lang.NoClassDefFoundError: io/github/bonigarcia/wdm/WebDriverManager

this error occurs when the application is unable to find required external libraries (such as Selenium WebDriver and WebDriverManager) at runtime. Even though the project builds successfully, the dependencies are not included inside the final JAR file, so Jenkins fails while executing the program.

To solve this issue, instead of running the application using a JAR file, we use Maven execution so that all dependencies defined in pom.xml are properly loaded during runtime execution.

This ensures that Selenium and WebDriverManager libraries are available when the program runs in Jenkins.

Replace the existing execution method with the following:

mvn exec:java -Dexec.mainClass="com.example.App"

Jenkins Pipeline Update:

```
stage('Run Application') { steps {

sh 'mvn exec:java -Dexec.mainClass="com.example.App"'
}
}
```

This stage is used to execute the Java application directly through Maven so that all dependencies defined in the project are included during runtime.

pom.xml Dependencies (Required Fix):

```
<dependency>

<groupId>org.seleniumhq.selenium</groupId>

<artifactId>selenium-java</artifactId>

<version>4.29.0</version>

</dependency>



<dependency>

<groupId>io.github.bonigarcia</groupId>

<artifactId>webdrivermanager</artifactId>

<version>5.7.0</version>

</dependency>
```

These dependencies are required to enable Selenium automation and automatic browser driver management during execution.

Changes made in App.java to solve the error:

To resolve the **NoClassDefFoundError (WebDriverManager missing at runtime)**, the following changes were ensured in App.java:

Added **WebDriverManager setup** to automatically handle ChromeDriver dependency: WebDriverManager.chromedriver().setup();

This line automatically downloads, sets up, and manages the correct version of ChromeDriver required for the installed Chrome browser. It removes the need to manually download or configure the driver path and ensures compatibility between Chrome and ChromeDriver during Jenkins execution.

Configured **ChromeOptions for Jenkins/Linux environment** to avoid browser crash in headless mode:

ChromeOptions options = new ChromeOptions(); options.addArguments("--headless=new"); options.addArguments("--no-sandbox"); options.addArguments("--disable-dev-shm-usage"); options.addArguments("--disable-gpu"); options.addArguments("--remote-allow-origins=*");

These options ensure that Chrome runs in a headless (no UI) mode suitable for Jenkins/Linux servers and prevent common issues like browser crash, sandbox errors, and memory limitations during execution.

Used **ChromeDriver with options** instead of normal driver: WebDriver driver = new ChromeDriver(options);

This ensures that the browser is launched with the configured ChromeOptions settings (such as headless mode and Jenkins-safe arguments), making the automation stable and suitable for execution in CI/CD environments like Jenkins.

Ensured proper **login automation flow** after driver initialization:

driver.get("https://www.saucedemo.com/"); driver.findElement(By.id("user-name")).sendKeys("standard_user");
driver.findElement(By.id("password")).sendKeys("secret_sauce");
driver.findElement(By.id("login-button")).click();

This performs the complete login automation flow in the SauceDemo application by opening the website, entering valid credentials, and clicking the login button to validate successful user authentication.

The following are the updated pom.xml, App.java, and Jenkinsfile: pom.xml

<project xmlns="http://maven.apache.org/POM/4.0.0"

xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0

http://maven.apache.org/xsd/maven-4.0.0.xsd">

<modelVersion>4.0.0</modelVersion>

<groupId>com.example</groupId>

<artifactId>MyMavenSeleniumApp01</artifactId>

<version>1.0-SNAPSHOT</version>

<packaging>jar</packaging>

<name>MyMavenSeleniumApp01</name>

<properties>

<maven.compiler.source>11</maven.compiler.source>

<maven.compiler.target>11</maven.compiler.target>

</properties>

```
<dependencies>
<!-- JUnit -->
<dependency>
<groupId>junit</groupId>
<artifactId>junit</artifactId>
<version>4.13.2</version>
<scope>test</scope>
</dependency>
<!-- Selenium -->
<dependency>
<groupId>org.seleniumhq.selenium</groupId>
<artifactId>selenium-java</artifactId>
<version>4.29.0</version>
</dependency>
<!-- WebDriverManager (FIX FOR YOUR ERROR) -->
<dependency>
<groupId>io.github.bonigarcia</groupId>
<artifactId>webdrivermanager</artifactId>
<version>5.7.0</version>
</dependency>
```

```
</dependencies>
<build>
<plugins>
<!-- Compiler Plugin -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler-plugin</artifactId>
<version>3.8.1</version>
<configuration>
<source>11</source>
<target>11</target>
</configuration>
</plugin>
<!-- Test Runner -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
<version>2.22.2</version>
</plugin>
<!-- Jar Plugin -->
```

```
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-jar-plugin</artifactId>
<version>3.0.1</version>
<configuration>
<archive>
<manifest>
<mainClass>com.example.App</mainClass>
</manifest>
</archive>
</configuration>
</plugin>
</plugins>
</build>
</project>
```

App.java

```
package com.example;
import io.github.bonigarcia.wdm.WebDriverManager; import
org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
```

```
import org.openqa.selenium.chrome.ChromeDriver; import
org.openqa.selenium.chrome.ChromeOptions;

public class App {

public static void main(String[] args) { try {

WebDriverManager.chromedriver().setup(); ChromeOptions options = new
ChromeOptions();

// Jenkins/Linux safe mode options.addArguments("--headless=new");
options.addArguments("--no-sandbox"); options.addArguments("--disable-dev-shm-
usage"); options.addArguments("--disable-gpu");

options.addArguments("--remote-allow-origins=*"); WebDriver driver = new
ChromeDriver(options);

// Open SauceDemo driver.get("https://www.saucedemo.com/");

// Maximize (works in non-headless too; safe to keep)
driver.manage().window().maximize();

// Login steps

driver.findElement(By.id("user-name")).sendKeys("standard_user");
driver.findElement(By.id("password")).sendKeys("secret_sauce");
driver.findElement(By.id("login-button")).click();

// Print after login

System.out.println("TITLE AFTER LOGIN: " + driver.getTitle());

// Close browser driver.quit();

} catch (Exception e) {  System.out.println("ERROR: " + e.getMessage());
e.printStackTrace();
}
}
}
```

Jenkinsfile:

```
pipeline {

agent any  // Use any available agent tools {

maven 'Maven'  // Ensure this matches the name configured in Jenkins
}

stages {

stage('Checkout') { steps {

git branch: 'master', url:
'https://github.com/Shweta311204/MyMavenSeleniumApp01.git'
}
}

stage('Build') { steps {

sh 'mvn clean package'  // Run Maven build
}
}

stage('Test') { steps {

sh 'mvn test'  // Run unit tests
}
}

stage('Run Application') { steps {

// Start the JAR application

sh 'mvn exec:java -Dexec.mainClass="com.example.App"'
}
}
}

post {
```

```
success {

echo 'Build and deployment successful!'
}

failure {

echo 'Build failed!'
}
}
}
```

Step 5: After making the changes, go back to **MyMavenSeleniumApp01** in Jenkins and click **Build Now**. The build should now complete successfully without any errors.

This happens because the dependencies (such as **Selenium and WebDriverManager**) are properly included in the project through Maven configuration, and the application is executed using mvn exec:java. So during runtime, Jenkins is able to load all required libraries correctly, and issues like **java.lang.NoClassDefFoundError** are resolved.